LLM Matrix Inversion

LLM Architecture, Training, and Evaluation for Structured Numeric Generation

Pranav Tikkawar

May 04, 2026

What this project is

Goal

Given a text description of a real-valued matrix A, the model should output a strictly formatted text block containing a numerically correct inverse A^{-1}.

This is a dual objective: formatting must be parseable since LLMs are prone to generating text that looks reasonable but is not actually correct, and the numbers must actually invert the matrix instead of providing a trivial solution via some pattern.

A prediction only counts as good if it survives both checks.

Typical failures: looping text, parseable but wrong numbers, and outputs that resemble A or $-A$ rather than A^{-1}.

3 methods for getting better LLM results

Initial Promping

Initial prompting is the most basic method: just give the model a clear instruction and hope for the best. This can work for simple tasks, but for complex tasks, it often leads to poor performance.

LoRA fine-tuning

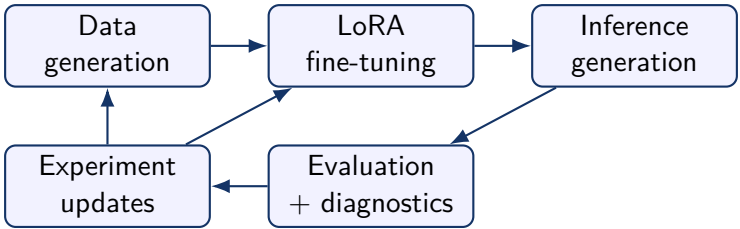
LoRA fine-tuning allows us to adapt the model's weights to our specific task without needing to update the entire model, which can be more efficient and effective for specialized tasks.

Multi-shot prompting

Multi-shot prompting provides the model with several examples of the desired output format and content, which can help it learn the pattern and improve its performance on the task.

I predominantly used a mix of LoRA fine-tuning and multi-shot prompting, since initial prompting alone was not sufficient to achieve good results on the matrix inversion task.

Overarching architecture



The point is rapid iteration: generate data, fine-tune adapters, evaluate, inspect failures, and change the next run.

The diagnostics are designed to tell whether the main problem is formatting, optimization, or numeric behavior.

Why LoRA and how it links to stochastics

LoRA idea

Freeze a pretrained weight matrix W and learn a low-rank update

$$W' = W + BA,$$

with small rank, instead of updating all weights.

LoRA reduces trainable parameters and memory by learning low-rank adapter matrices rather than fully updating pretrained weights [1].

Training is still stochastic optimization: mini-batches produce noisy gradient estimates, but those updates now live in a constrained low-rank subspace [2].

For this project, that means a key question is whether the adapter has enough capacity to learn both strict syntax and precise numerics.

How I judge success

Primary metric

Use the inversion residual

$$\|A\hat{X} - I\|_F$$

as the main score.

Near zero means the output behaves like an inverse.

Large values indicate that even a well-formatted answer is numerically wrong.

I also track parse success, stop reason, token count, right residual $\|\hat{X}A - I\|_F$, and whether $\hat{X}$ is closer to A or $-A$ than to A^{-1}.

Three output categories

Most of the outputs I get fall into one of three categories:

No output / unreadable

Looping or truncated generations.

Multiple `Inverse:` headers, extra text, or broken layout.

Parser fails, so we cannot even compute $A\hat{X}$ or residuals.

Looks like $-A$

Output is a clean matrix but entries are roughly the negative of A.

$\|\hat{X} + A\|_F$ is smaller than $\|\hat{X} - A^{-1}\|_F$.

$\|A\hat{X} - I\|_F$ remains large, so this is not an inverse.

Looks like $1/A$ (elementwise inverse)

Clean, single matrix in the right format.

Output is still incorrect as it resembles the elementwise inverse rather than the true matrix inverse.

Unfortunately, the model does not seem to ever produce a true inverse on the eval set, but it does produce outputs in the last two categories that are close enough to parse and evaluate, which gives us some signal to work with.

Results and Potential Symptoms

The first output category (no output/unreadable) is the most common among the no-shot, pre-trained model, and nearly disappears with LoRA fine-tuning and multi-shot prompting, which suggests that the model is learning to follow the output format and stop appropriately.

With low training and low data regimes, the second category (looks like $-A$) is more common, which could indicate that the model is picking up on some superficial pattern in the input matrix rather than learning the true mathematical relationship.

The third category (looks like $1/A$) appears more frequently with multi-shot prompting, which might suggest that the model is learning to produce a matrix that is related to the input but is not actually the correct inverse, possibly due to overfitting to the examples provided in the prompt rather than generalizing the underlying mathematical concept.

While the model never produces a correct inverse, the shift in output categories with different training methods provides insight into how the model is learning and where it might be going wrong, which can inform our next steps in improving its performance on this task.

Next steps

More data: generate more and harder matrices to better cover the space the model needs to invert.

Better parsing: tighten the output format and add guardrails so looping and extra text are less likely.

More multi-shot prompting: show more varied correct inverses so the model sees the pattern clearly.

Longer, targeted training: run longer LoRA training on curated hard examples once the basics look solid.

Main message

The project is really about structured numeric generation, not just text generation.

LoRA makes the experimentation loop fast enough to test data, rank, decoding, and formatting hypotheses.

The central empirical question is where the model fails first: parsing, looping, or true inversion quality.

Questions

Edward J. Hu et al.
LoRA: Low-Rank Adaptation of Large Language Models.
arXiv:2106.09685, 2021.

Hugging Face.
PEFT documentation: LoRA.
https://huggingface.co/docs/peft/v0.8.0/package_reference/lora.